Scientific Computing WS25/26

Common Collections

Common collections

Most data types hold a single value, but collections can store many. Unlike arrays and tuples, their data lives on the heap, so the size doesn’t need to be known at compile time and can grow or shrink while the program runs.

Most used collections used in Rust are:

- Vectors
- Strings
- Hash maps

Storing Lists of Values with Vectors

What is a vector?

Usually denoted as **Vec <T>**, let you store many values together in one place, with all the items kept next to each other in memory.

To create a new empty vector, we call the function **Vec::new**

```
let v: Vec<i32> = Vec::new();
```

Updating a vector:

- We do that by using the **push** method
- We use **mut** to make the value of any variable changeable.

```
let mut v = Vec::new();

v.push(5);
v.push(6);
v.push(7);
v.push(8);
```

Reading Elements of Vectors

You can access values in a vector in two ways:

- using an **index**
- calling the **`get`** method.

```rust
let v = vec![1, 2, 3, 4, 5];

let third: &i32 = &v[2];
println!("The third element is {third}");

let third: Option<&i32> = v.get(2);
match third {
    Some(third) => println!("The third element is {third}"),
    None => println!("There is no third element."),
}
```

Example:

```
let v = vec![1, 2, 3, 4, 5];

let does_not_exist = &v[100];
let does_not_exist = v.get(100);
```

If you run this code, the first `[]` access will panic because it tries to read an element that doesn't exist.

Using **`get`** with an out-of-range index returns **`None`** instead of panicking. This is useful when out-of-bounds access might happen, and you can handle either **`Some(&element)`** or **`None`** in your code.

Storing Lists of Values with Vectors

```
let mut v = vec![1, 2, 3, 4, 5];

let first = &v[0];

v.push(6);

println!("The first element is: {first}");
```

- When you have a valid reference, the borrow checker makes sure all borrowing rules are followed.

- **Remember**: you can't have mutable and immutable references to the same data at the same time.

- The example *seems* like it should work, but it doesn't because of how vectors store data.

```
$ cargo run
   Compiling collections v0.1.0 (file:///projects/collections)
error[E0502]: cannot borrow `v` as mutable because it is also borrowed as immutable
 --> src/main.rs:6:5
  |
4 |     let first = &v[0];
  |                 - immutable borrow occurs here
5 |
6 |     v.push(6);
  |     ^^^^^^^^^ mutable borrow occurs here
7 |
8 |     println!("The first element is: {first}");
  |                                     ------- immutable borrow later used here

For more information about this error, try `rustc --explain E0502`.
error: could not compile `collections` (bin "collections") due to 1 previous error
```

Iterating Over the Values in a Vector:

```
let v = vec![100, 32, 57];
for i in &v {
    println!("{i}");
}
```

- Use a `for` loop to iterate over a vector of `i32` and print each element by taking an immutable reference.

- Use a `for` loop to iterate over mutable references in a vector and modify each element (e.g., adding 50 to every value).

```
let mut v = vec![100, 32, 57];
for i in &mut v {
    *i += 50;
}
```

Using an Enum to Store Multiple Types:

```
enum SpreadsheetCell {
    Int(i32),
    Float(f64),
    Text(String),
}

let row = vec![
    SpreadsheetCell::Int(3),
    SpreadsheetCell::Text(String::from("blue")),
    SpreadsheetCell::Float(10.12),
];
```

- Vectors can't store mixed types, but an enum lets you wrap different kinds of values under one shared type.
- By defining an enum with variants for each needed type, you can store mixed data (like ints, floats, and strings) in a single vector.
- Rust requires vectors to have a known, fixed set of types at compile time. Enums + `match` ensure all cases are handled, but if types aren't known in advance you'll need trait objects.

Dropping a Vector Drops Its Elements:

Like any other struct, a vector is freed when it goes out of scope.

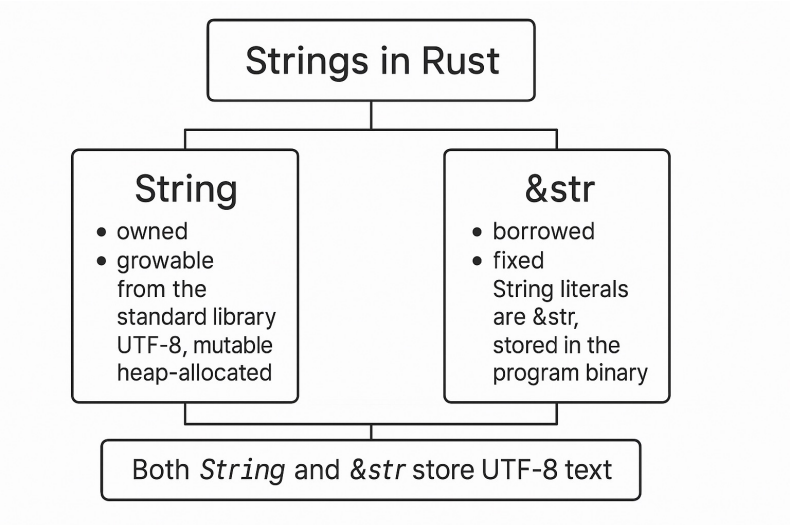
```
{
    let v = vec![1, 2, 3, 4];

    // do stuff with v
} // <- v goes out of scope and is freed here
```

Storing UTF-8 Encoded Text with Strings

What is a string?

- Rust has two main string types: **`String`** (owned, growable) and **`&str`** (borrowed, fixed).
- String literals are **`&str`**, stored in the program binary.
- `String` comes from the standard library and is UTF-8, mutable, and heap-allocated.
- Both **`String`** and **`&str`** store UTF-8 text and are widely used in Rust.



Creating a New String:

- Many `Vec<T>` operations also work on `String` because `String` is essentially a byte-vector wrapper with extra rules. For example, both use `new()` to create a new instance.

```
let mut s = String::new();
```

- This line makes a new empty `String` named `s` that we can fill later. If you want to start with initial text, use `to_string`, which works on any type implementing `Display` (including string literals).

```
let data = "initial contents";

let s = data.to_string();

// The method also works on a literal directly:
let s = "initial contents".to_string();
```

- This code creates a string containing **initial contents**

Updating a String:

A `String` can grow and change just like a `Vec<T>`. You can add more text to it, and you can combine strings using the `+` operator or the `format!` Macro.

We can grow a `String` by using the `push_str` method to append a string slice.

```
let mut s = String::from("foo");
s.push_str("bar");
```

```
let mut s1 = String::from("foo");
let s2 = "bar";
s1.push_str(s2);
println!("s2 is {s2}");
```

Concatenation with the `+` Operator or the `format!` Macro:

```
let s1 = String::from("Hello, ");
let s2 = String::from("world!");
let s3 = s1 + &s2; // note s1 has been moved here and can no longer be used
```

```
fn add(self, s: &str) -> String {
```

Storing UTF-8 Encoded Text with Strings

- When you need to concatenate(join) several strings, using the **+** operator becomes messy and hard to read.

```
let s1 = String::from("tic");
let s2 = String::from("tac");
let s3 = String::from("toe");

let s = s1 + "-" + &s2 + "-" + &s3;
```

- For more complex string building, it's better to use the **`format!`** macro.

```
let s1 = String::from("tic");
let s2 = String::from("tac");
let s3 = String::from("toe");

let s = format!("{s1}-{s2}-{s3}");
```

Indexing into Strings:

In many languages you can access string characters by index, but in Rust this doesn't work—trying to index into a `String` results in an error.

```
let s1 = String::from("hi");
let h = s1[0];
```

```
$ cargo run
   Compiling collections v0.1.0 (file:///projects/collections)
error[E0277]: the type `str` cannot be indexed by `{integer}`
 --> src/main.rs:3:16
  |
3 |     let h = s1[0];
  |                ^ string indices are ranges of `usize`
  |
  = note: you can use `.chars().nth()` or `.bytes().nth()`
          for more information, see chapter 8 in The Book: <https://doc.rust-lang.org/bo
  = help: the trait `SliceIndex<str>` is not implemented for `{integer}`
          but trait `SliceIndex<[_]>` is implemented for `usize`
  = help: for that trait implementation, expected `[_]`, found `str`
  = note: required for `String` to implement `Index<{integer}>`

For more information about this error, try `rustc --explain E0277`.
error: could not compile `collections` (bin "collections") due to 1 previous error
```

Internal Representation:

```
let hello = String::from("Hola");
```

- Here, `len` is 4 because the UTF-8 bytes for `"Hola"` take up four bytes—one for each character.

```
let hello = String::from("Здравствуйте");
```

- Rust reports 24 bytes because each character in “Здравствуйте” takes 2 bytes in UTF-8. This means byte indexes don’t line up with Unicode characters

Bytes and Scalar Values and Grapheme Clusters:

Another thing about UTF-8 is that Rust strings can be viewed in three ways:

- bytes,
- Unicode scalar values and
- grapheme clusters (what we typically think of as letters).

For example, the Hindi word “नमस्ते” in Devanagari is stored as a u8 byte vector that looks like this:

```
[224, 164, 168, 224, 164, 174, 224, 164, 184, 224, 165, 141, 224, 164, 164,
224, 165, 135]
```

```
['न', 'म', 'स', '्', 'त', 'े']
```

```
["न", "म", "स्", "ते"]
```

Slicing Strings

Indexing strings is tricky because it's unclear what a single index should return—a byte, a character, or a grapheme cluster. So if you need to slice a string by position, Rust requires you to be explicit. Instead of using `[]` with one number, you must use a byte range to create a slice.

```
let hello = "Здравствуйте";

let s = &hello[0..4];
```

Methods for Iterating Over Strings

The best way to work with parts of a string is to be clear about whether you want characters or bytes. Use **`chars`** to get individual Unicode characters (e.g., `"Зд"` **`.chars()`** returns two **`char`** values), or use **`bytes`** to get the raw UTF-8 bytes if that's what you need.

```
for c in "Зд".chars() {
    println!("{c}");
}
```

```
З
д
```

```
for b in "Зд".bytes() {
    println!("{b}");
}
```

```
208
151
208
180
```

Storing Keys with Associated Values in Hash Maps

Storing Keys with Associated Values in Hash Maps

A **hash map** (`HashMap<K, V>`) stores key–value pairs using a hashing function. Many languages have similar structures, often called dictionaries, maps, or hash tables.

Hash maps are great when you want to look up data by a **key** instead of a numeric index. For example, in a game you could map each team's name to its score and retrieve the score by giving the team's name.

Creating a New Hash Map

One way to create an empty hash map is to use **new** and to add elements with **insert.**

```
use std::collections::HashMap;

let mut scores = HashMap::new();

scores.insert(String::from("Blue"), 10);
scores.insert(String::from("Yellow"), 50);
```

Accessing Values in a Hash Map

Use the `get` method with a key to retrieve a value from a hash map.

```
use std::collections::HashMap;

let mut scores = HashMap::new();

scores.insert(String::from("Blue"), 10);
scores.insert(String::from("Yellow"), 50);

let team_name = String::from("Blue");
let score = scores.get(&team_name).copied().unwrap_or(0);
```

```
Yellow: 50
Blue: 10
```

```
use std::collections::HashMap;

let mut scores = HashMap::new();

scores.insert(String::from("Blue"), 10);
scores.insert(String::from("Yellow"), 50);

for (key, value) in &scores {
    println!("{key}: {value}");
}
```

Hash Maps and Ownership

Types that implement Copy (like `i32`) get copied into the hash map. Owned types like `String` are moved, making the hash map their new owner.

```
use std::collections::HashMap;

let field_name = String::from("Favorite color");
let field_value = String::from("Blue");

let mut map = HashMap::new();
map.insert(field_name, field_value);
// field_name and field_value are invalid at this point, try using them and
// see what compiler error you get!
```

Updating a Hash Map

A hash map can grow, but each key can only have one value at a time (though different keys can share the same value).

When updating a key, you must decide what to do if it already has a value:

- replace the old value,
- keep the old value and ignore the new one, or
- combine the two.

Updating a Value Based on the Old Value

- You can update values in a hash map based on their existing value.
- `entry().or_insert(0)` inserts `0` for new words and returns a mutable reference.
- Incrementing this reference lets you count word occurrences safely.
- Hash map iteration order is arbitrary, but Rust's borrowing rules ensure safe updates.

```rust
use std::collections::HashMap;

let text = "hello world wonderful world";

let mut map = HashMap::new();

for word in text.split_whitespace() {
    let count = map.entry(word).or_insert(0);
    *count += 1;
}

println!("{map:?}");
```

Thank You